

What the Investigator Agent hands off

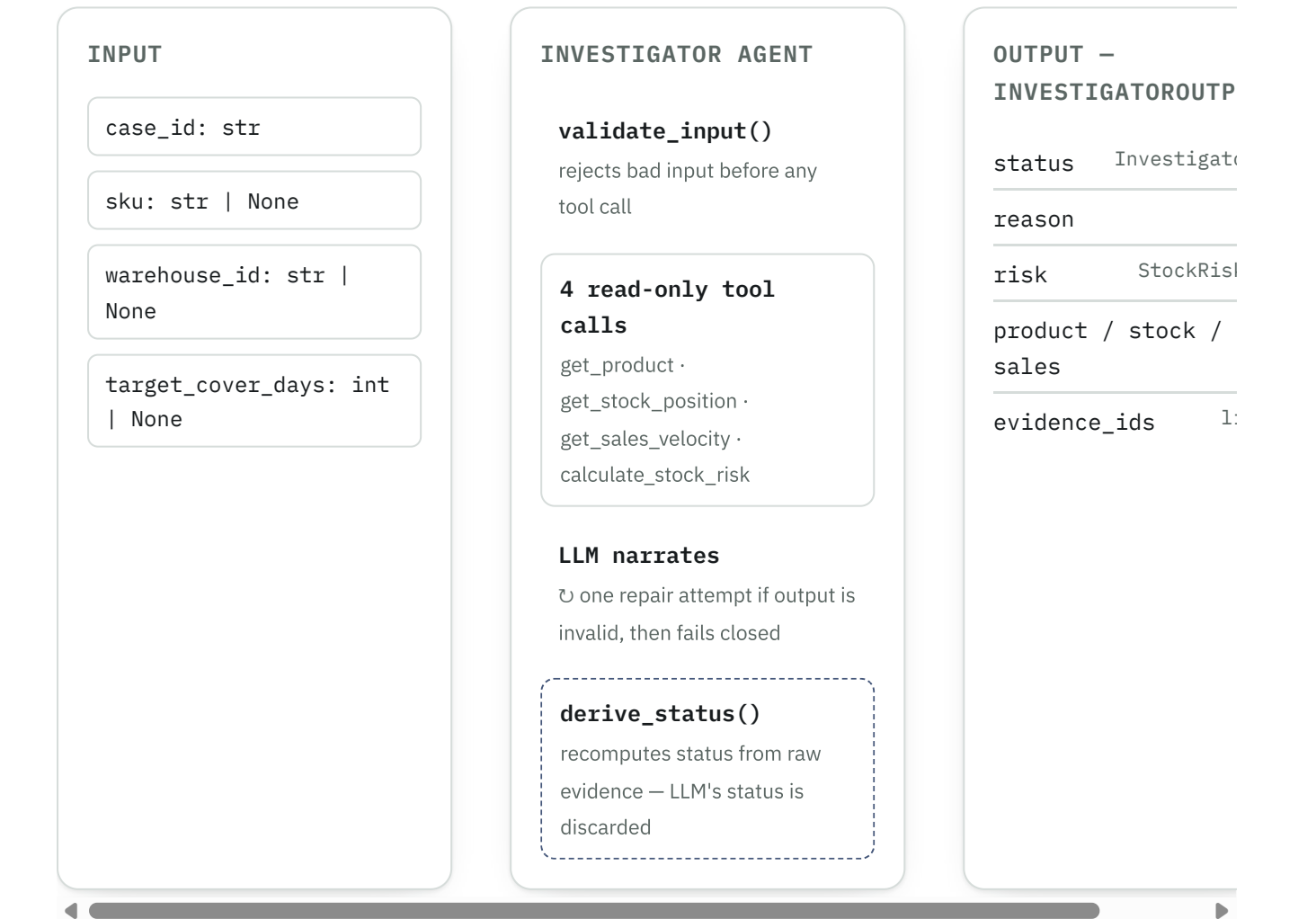
This is the contract other agents build against: what goes in, what comes out, the four shapes the output can take, and exactly which fields the Sourcing Agent will consume next.

✓ Built & tested – `submission/agents/investigator.py`

01

Input → pipeline → output

The LLM only decides *which tools to call* and *how to phrase the reason*. Everything else — including whether its own answer is even well-formed — is checked in code.



02

Four possible shapes

Every case lands in exactly one of these. Only `AT_RISK` continues to the Sourcing Agent — the other three are terminal.

NO_ACTION

Coverage is healthy. Terminal.

```
product · stock · sales · risk ✓  
evidence_ids ✓  
missing_field, error_code
```

NEEDS_INFORMATION

Bad input, or sales history too thin to trust.
Terminal.

```
reason, error_code ✓  
(bad input) everything else None  
(thin history) product · stock · sales ✓,  
risk None
```

BLOCKED

SKU/warehouse unresolvable, or snapshot stale. Terminal.

```
product · stock · sales ✓  
risk ✓ only for DATA_STALE  
risk None for NOT_FOUND / INACTIVE
```

AT_RISK

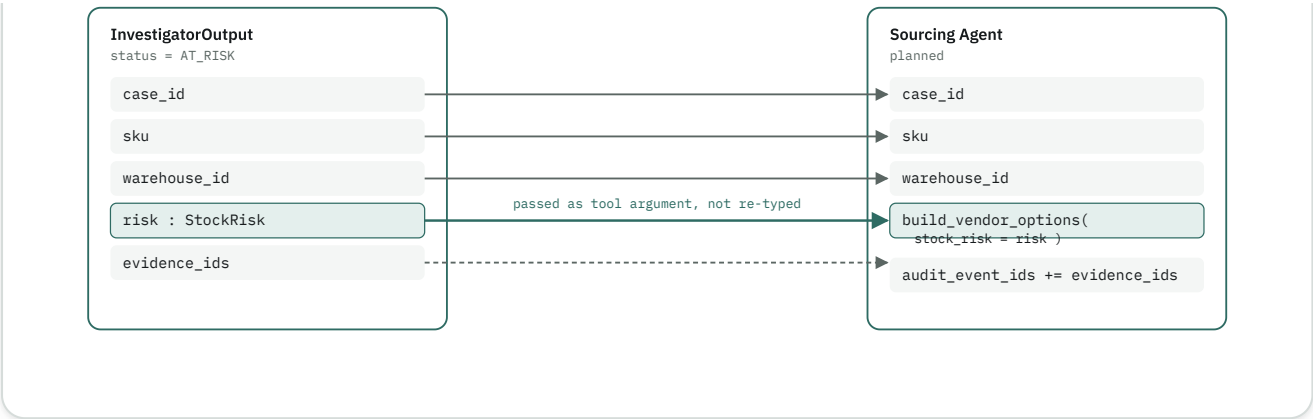
Coverage is short of target.

```
product · stock · sales · risk ✓ (all  
populated)  
evidence_ids ✓ (3 ids)  
→ continues to Sourcing Agent
```

03

Handoff to the Sourcing Agent

Only the `AT_RISK` exit is wired forward. The graph orchestrator passes these fields straight through — nothing is re-derived or re-typed by hand.



Claim: `case_id` , `sku` , and `warehouse_id` pass through unchanged; `risk` is handed straight into `build_vendor_options(stock_risk=...)` — the same deterministic tool the Sourcing Agent calls — so the stockout date and available units are never re-typed by an LLM.

04

Field reference

Full shape of `InvestigatorOutput` (`submission/contracts.py`).

FIELD	TYPE	POPULATED WHEN
<code>case_id</code>	<code>str</code>	Always
<code>sku</code> / <code>warehouse_id</code> / <code>target_cover_days</code>	<code>Optional</code>	Echoed from input, even if empty when invalid
<code>status</code>	<code>InvestigatorStatus</code>	Always — the field even if the branch decision reached
<code>reason</code>	<code>str</code>	Always — LLM prose, even if empty to show a human as-sessment
<code>missing_field</code>	<code>Optional[str]</code>	Only on <code>NEEDS_INFORMATION</code> if the field was missing from bad input
<code>error_code</code>	<code>Optional[ErrorCode]</code>	<code>NOT_FOUND</code> , <code>INACTIVE</code> , <code>DATA_STALE</code> , <code>INSUFFICIENT_DATA</code> , <code>INVALID_INPUT</code> (reported by the LLM)

		exhausted)
product / stock / sales	Optional[...]	Populated once the corresponding tool has been called
risk	Optional[StockRisk]	Only when product, stock and sales all succeed this is the object build_vendor_option expects as stock r